Anotações para a prova

Competências Angular

1. Compreender Arquitetura Angular e seus comandos - aula 1 (2hrs)
2. Elaborar componentes (templates e bindings) - Aula 1 e 2 (6 hrs)
3. Produzir formulários reativos utilizando forms - Aula 3 (1hr)
4. Utilizar o conceito de SPA (single page application) - Aula 3 (2hrs)
5. Compreender funcionamento de componentes 'standalone' - Aula 3 (1hr)

```
@Component({

    selector: 'app-meu-componente',

    standalone: true,

    imports: [CommonModule, OutroComponente],

    template: `...`

})
export class MeuComponente { }
```

6. Aplicar ferramentas de otimização de renderização reativa (signals) - Aula 4 (4 hrs)
7. Aplicar ferramentas de operações assíncronas (observables) - Aula 5 (2 hrs)
8. Utilizar modulação para fazer requisições https - Aula 5 (2hrs)

SE DER TEMPO*

9. Implementar conexões frontend e backend com serviços de websocket (RPLACE) - Aula 6 e 7 (8 hrs)

10. Implementar conexões frontend e backend com serviços de messageria

Comandos:

- Instalar Angular: `npm  install -g @angular/cli@16`
- Desinstalar Angular: npm uninstall -g @angular/cling g service domain/api
- Ajuda: `ng --help`
- Verificar versão: `ng version`
- Criar um novo projeto: `ng new “Nome do Projeto”`
- Criar um novo componente: `ng generate component components/input`
- Criar componente (standalone): `ng g c features/board/components/canvas --standalone`
- Criar um Guard: `ng g guard domain/AuthGuard`

Two way binding:

```
import { FormsModule } from '@angular/forms'

[(ngModel)]="text"
```

Signals (Otimização de Renderização)

mostrar algo na tela que seja atualizado - integra

```
// rastreabilidade de uma variável
// monitorar alterações
export class App {
    protected readonly title = signal('rplace');
}
```

O que é? Signals são a nova forma do Angular gerenciar "Estado" (variáveis que mudam e precisam atualizar a tela). Quando um Signal muda, o Angular atualiza exatamente e apenas a parte do HTML que usa aquele Signal. É brutalmente mais rápido e performático que variáveis comuns.

Cenário: O usuário escolhe uma cor na paleta de cores. Queremos armazenar a "cor atual" que ele está segurando na mão para pintar.

```
color-state.service.ts (Serviço para guardar o estado)
import { Injectable, signal } from '@angular/core';
@Injectable({ providedIn: 'root' })
export class ColorStateService {
 // Criamos um Signal. Valor inicial é preto.
 // Ele é "writable", ou seja, podemos alterar o valor del
e.
 public selectedColor = signal('#000000');
 // Função para mudar a cor
 changeColor(newColor: string) {
 this.selectedColor.set(newColor); // Atualiza o signal
 }
}
```

board.component.ts (O Pai - usando o signal)

```
import { Component, inject } from '@angular/core';
import { ColorStateService } from '../../core/services/colo
r-state.service';
@Component({
 selector: 'app-board',
 standalone: true,
 template: `

## Cor selecionada atualmente: {{ colorService.selectedColo
r()
}}

Pegar Vermelho
 `
})
export class BoardComponent {
```

```
 // inject() é a forma moderna de injetar serviços no Angu
lar Standalone
 colorService = inject(ColorStateService);
 mudarPraVermelho() {
 this.colorService.changeColor('#FF0000');
 }
}
```

Quando usar: Para qualquer variável cujo valor vai mudar na tela ao longo do tempo de forma síncrona (como itens selecionados, contadores, configurações Angular 9
de UI).

Stand Alone Component:

O que é um Standalone Component? É um componente que não precisa ser declarado em um arquivo `module.ts` (`@NgModule`). Ele importa suas próprias dependências, tornando o código muito mais limpo.

- Property Binding [propriedade]="valor" :: Envia um dado do TS para uma propriedade do HTML.
- Event Binding (evento)="funcao()" Escuta uma ação do HTML (como um clique) e chama uma função no TS.
- @Input: Permite que um componente "Pai" passe dados para um componente "Filho".
- @Output: Permite que o "Filho" avise o "Pai" que algo aconteceu (usando o EventEmitter).

Observables, Requisições HTTP, WebSockets e Mensageria

- **Http Client:** Usado para pegar o estado inicial do quadro (o banco de dados retorna uma foto de como o quadro está agora). Requisição única (bate no servidor e a conexão morre).
- **WebSockets / Mensageria:** Usado para comunicação em tempo real. O servidor e o front-end ficam com um "tubo" aberto. Se Joãozinho lá no Japão pintar um pixel, o servidor avisa o nosso frontend na mesma hora, sem a gente precisar dar F5.

- **Observables:** Representam um fluxo de dados no tempo. Perfeitos para lidar com respostas HTTP e mensagens do WebSocket.

  O `.pipe()` é um método do RxJS disponível em todos os Observables. Ele funciona como um mecanismo de montagem. Imagine uma linha de produção
  em uma fábrica: o `pipe` é a esteira rolante.

ponte é feita com o httpclient

envia pacotes entre front e back → subscribe envia

observable monitora esse processo e cuida dos acessos

pipe fecha a ponte

```
<input class="input" (input)="changeText($event)" />
```


```
<input class="input" (input)="changeText($event)" />
```


quando for () o valor vai ir do html para o ts ou seja output e quando é [ ] vai do ts para o html ou

seja imput

Organização das pastas

— App

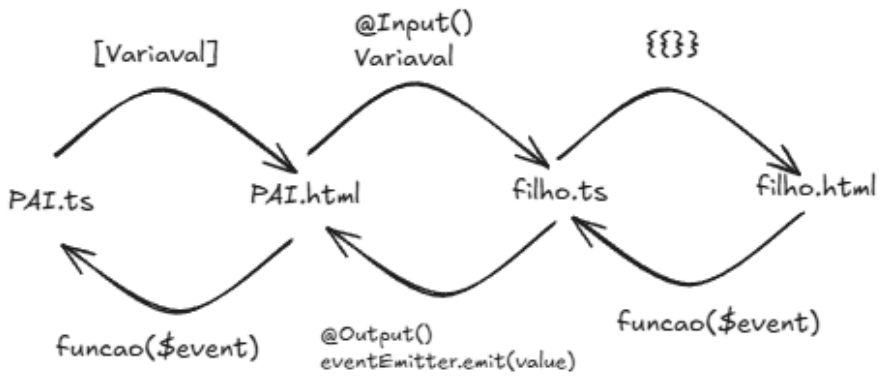
— Features

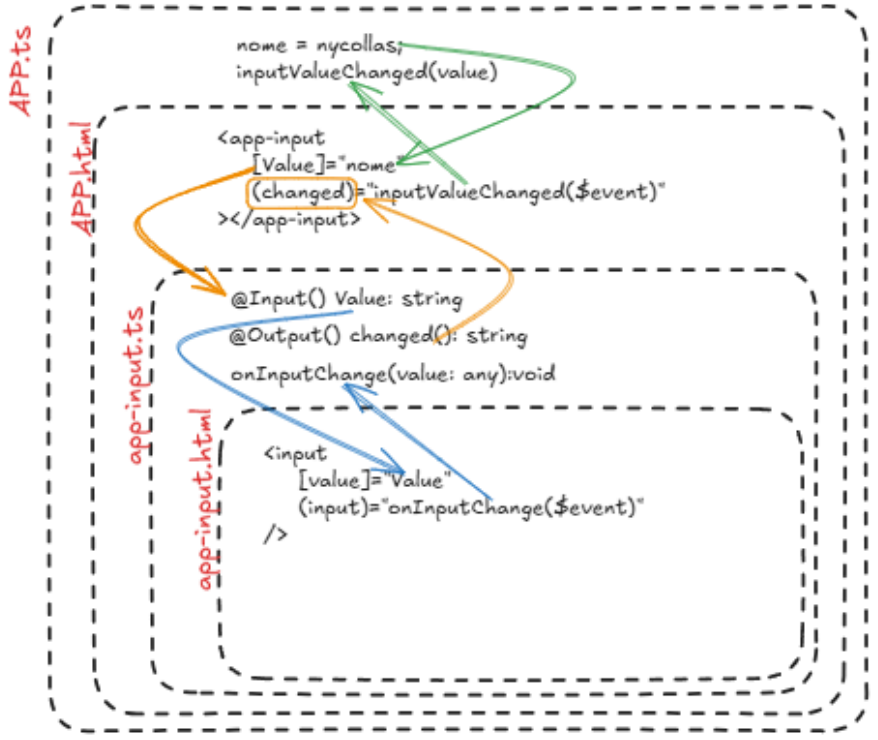
— Main

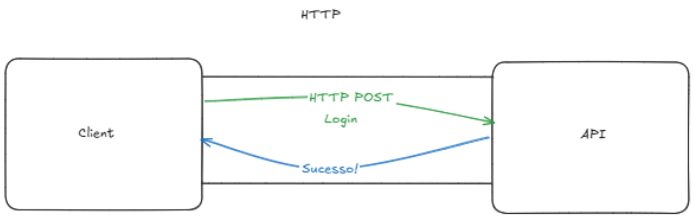
— NotFo

— Shared

— Header


[Variaval]
@Input()
Variaval
{{}}
PAI.ts
PAI.html
filho.ts
filho.html
funcao($event)
@Output()
eventEmitter.emit(value)
funcao($event)


APP.ts
nome = nycollas;
inputValueChanged(value)
APP.html
<app-input
[Value]="nome"
(changed)="inputValueChanged($event)"
></app-input>
app-input.ts
@Input() Value: string
@Output() changed(): string
onInputChange(value: any):void
app-input.html
<input
[value]="Value"
(input)="onInputChange($event)"
/>



Verificação projeto

1. Setup e Arquitetura (Competências 1, 4 e 5)

☐ Criação do Projeto: Garantiu que o projeto é Standalone (Angular 14+ ou 17+)

☐ Estrutura de Pastas: Criou as pastas features (lógica de tela), shared (reutilizáveis), core (serviços e guards) e domain (interfaces/types).

☐ Rotas (SPA): Configurou o arquivo app.routes.ts com as rotas para 'Login' e 'Board'

☐ Standalone Components: Verificou se todos os componentes têm a propriedade standalone: true e se os imports necessários (CommonModule, FormsModule) estão dentro do array imports: [] de cada componente.

2. Fundamentos e Comunicação entre Componentes (Competência 2)

☐ Interfaces (Types): Criou a interface no diretório domain .

☐ @Input(): No componente Filho, criou os inputs para receber dados

☐ @Output() / EventEmitter: Criou o evento para avisar o componente pai.

☐ Property & Event Binding:

- [ ] No HTML do filho: usou [style.background-color] para a cor.
- [ ] No HTML do filho: usou (click) para disparar a função de emissão.
- [ ] No HTML do pai: usou (nomeDoEvento)="minhaFuncao($event)" para capturar o
clique.

3. Formulários e Estado Local (Competências 3 e 6)

- [ ] Reactive Forms: No componente de Login, injetou o FormBuilder e criou o FormGroup com validações (Validators.required).
- [ ] Binding de Form: Conectou o [formGroup] no HTML e os formControlName nos
inputs.
- [ ] Signals: Criou um serviço (ou variável no componente) usando signal<string>('#000000') para gerenciamento.
- [ ] Leitura de Signals: Garantiu que no HTML a leitura é feita com parênteses: {{ meuSignal() }} .

4. Comunicação Assíncrona e HTTP (Competências 7 e 8)

- [ ] HttpClient: Injetou o HttpClient no serviço principal.
- [ ] Observables (GET): Criou uma função que retorna um Observable da API (ex: getBoard()).
- [ ] Subscription: No ngOnInit do componente pai, deu um .subscribe() no método do serviço para preencher o quadro inicial.
- [ ] Pipes (RxJS): Usou operadores como map ou tap para tratar os dados antes de chegarem ao componente.

5. Tempo Real e Mensageria (Competências Opcionais 9 e 10)

- [ ] WebSocket Connection: Usou o webSocket do rxjs/webSocket para abrir a conexão no serviço.
- [ ] Escuta Ativa: Criou um Observable (ou Subject) que fica "ouvindo" as mensagens que chegam do servidor sobre novos pixels pintados.

☐ Atualização Reativa: Ao receber um dado via WebSocket, utilizou o método .update() do seu Signal de pixels para atualizar a tela sem recarregar nada.

Segurança e Controle (Extras mencionados)

☐ Auth Guard: Criou um arquivo .guard.ts que implementa CanActivateFn para proteger a rota do Board.

☐ Service Logic: Garantiu que a lógica de "pintar" (fazer o POST ou mandar pro Socket) está dentro do serviço, e não dentro do componente (princípio de responsabilidade única).